

External Data Dashboard: Crypto Price Tracker

Intern Take-Home Assignment | Soumya Ranjan Panda | Time spent: 2 to 3 hours

1. What the assignment asks for

Build a small pipeline that pulls data from a free public API on a schedule, stores every pull as a timestamped snapshot in a real database, computes derived metrics from the history (change since last pull, rolling average, min/max), serves it through GET /latest and GET /history, and shows it on a dashboard with the current value, metrics and a line chart. It must survive a slow or failing API, come with a README and include automated tests, especially for the metric math. The evaluation looks at pipeline separation, data modeling, correctness, code quality and communication.

2. What I built

A Python app that tracks **Bitcoin, Ethereum and Solana prices in USD** from the CoinGecko public API. Stack: **FastAPI** for the API, **SQLite** for storage, and a **single HTML page with Chart.js** for the dashboard. The whole project is around 250 lines so every part is easy to read and change.

I picked CoinGecko over Frankfurter because crypto prices change every minute, so repeated pulls actually show movement. Frankfurter updates once a day, which would give 0% change on almost every pull.

3. Pipeline structure

Stage	File	What it does
Pull + store	app/pull.py	Calls CoinGecko with a 10s timeout and 3 retries, then saves the pull attempt and the prices
Compute	app/metrics.py	Pure functions for change, rolling average, min/max and alert. No database or web code inside
Serve	app/main.py	FastAPI routes, background scheduler and a small in-memory cache
Display	app/static/index.html	Dashboard with metric tiles, status banner, chart and test buttons

4. Data model

Two tables. Nothing is ever overwritten, so history keeps growing and failed attempts stay visible.

Table	Columns	Purpose
pulls	id, pulled_at, status (ok / failed), error	One row per pull attempt, success or failure
prices	pull_id, coin, price, pulled_at	One row per coin per successful pull, indexed on (coin, pulled_at)

Timestamps are UTC in the format YYYY-MM-DD HH:MM:SS, so plain string comparison works for date filtering.

5. Derived metrics

Metric	Formula	Edge case
Change since last pull	last - prev, and (last - prev) / prev x 100	Needs 2 pulls, % is empty if prev is 0
Rolling average	Mean of the last N pulls (N = 5, configurable)	Uses all points if fewer than N
Min / Max	Over all stored history for that coin	Empty when no data yet
Alert	abs(% change) >= ALERT_PCT (default 2%)	Off until there are 2 pulls

6. Handling a flaky API

- Every request has a **10 second timeout** and is **retried 3 times** with a 2 second wait.
- If it still fails, the attempt is saved with status **failed** and the error message. The app keeps running.
- A partial response (a coin missing) counts as a failed pull, so half a snapshot is never stored.
- The dashboard shows a red banner: when the last pull failed, why, and the time of the last good data it is still showing. With no data at all, it tells the user to click Pull now.

7. Dashboard and testing UI

The page has a coin switcher, tiles for current price, change, rolling average, min, max and pulls stored, a big-move alert and a line chart with a From/To date filter. For testing there are two buttons: **Pull now** triggers a real pull, and **Simulate failed pull** records a fake failure so the error state can be checked without breaking the network. The page refreshes every minute so scheduled pulls appear on their own.

8. API

Endpoint	Returns
GET /latest?coin=bitcoin	Latest price, all derived metrics, alert flag and details of the last pull attempt
GET /history?coin=&start=&end=	Stored time series. A date-only end includes that whole day
POST /pull (?fail=true)	Runs a pull now. fail=true fakes a failure for testing
GET /coins	List of tracked coins

9. Tests

14 tests with pytest, all passing. **test_metrics.py** checks the math directly: % change divides by the previous value not the current one, rolling average takes the last N and not the first N, fewer than N points, empty lists, a zero previous value, min/max and alerts in both directions. **test_api.py** uses a temporary database and a fake fetch to check /latest after real pulls, that a failed pull keeps the last good data, the inclusive end-date filter, and a 404 for unknown coins.

10. Decisions left open, and my call

- **Scheduling:** a background thread pulls every 10 minutes (PULL_EVERY_MIN, 0 turns it off). Manual pulls work from the UI or with `py -m app.pull`. With more time I would move this to cron, APScheduler or a separate worker so it does not run twice when the app is scaled.
- **Rolling window:** last 5 pulls, set through ROLLING_WINDOW. Short enough to react, long enough to smooth single spikes.
- **Failure display:** a clear red banner plus the last good data, instead of an empty page.
- **Chart:** Chart.js from a CDN. No build step needed.

11. Stretch goals

All three are done: **multiple entities** (three coins with a switcher), **threshold alert** (ALERT_PCT) and **caching** (computed metrics cached per coin and cleared after every pull, while last-pull status is always read fresh).

12. Known limits and next steps

- /latest loads a coin's full history to find min/max. Fine for thousands of rows; for more I would compute it in SQL and fetch only the last N rows.
- On free hosting like Render the disk resets on redeploy, so SQLite history is lost. A real deploy would use a persistent disk or Postgres.
- The simulate-failure option is open and should sit behind auth or be removed in production.

13. How to run and deploy

`py -m pip install -r requirements.txt`, then `py -m uvicorn app.main:app --reload`, and open `http://localhost:8000` (API docs at /docs). Tests: `py -m pytest`. A Dockerfile and render.yaml are included for deploying on Docker or Render.

14. Use of AI tools

I used an AI coding assistant (Claude) to speed up writing the code and this report. I reviewed every file, ran the tests and checked the metric formulas by hand. I understand how each part works, including the schema, retry logic, caching and date filtering, and I can explain, debug and modify any of it as per new requirements in the follow-up discussion.